

## Middleware, Custom Middleware in .net explain experience interview level

In an experienced-level interview, don't describe middleware only as "something between request and response." Explain it as the **ASP.NET Core HTTP processing pipeline**, how ordering affects behavior, when to build custom middleware, and what production concerns you consider.

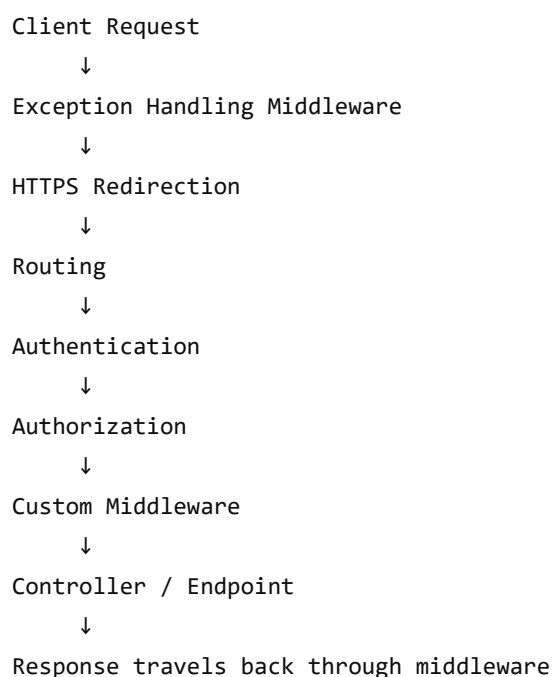
### What is Middleware in ASP.NET Core?

Middleware is a software component that participates in the ASP.NET Core **HTTP request/response pipeline**.

Each middleware can:

- Inspect or modify the incoming request.
- Execute logic before the next middleware.
- Call the next middleware using `await next(context)`.
- Execute logic after downstream middleware completes.
- Modify the outgoing response.
- Stop the pipeline completely, called **short-circuiting**.

Conceptually:



↑  
Client Response

A strong interview statement is:

Middleware forms a chain of components around every HTTP request. Each component decides whether to continue the pipeline, perform work before or after the next component, or short-circuit the request completely.

## How Middleware Works Internally

Consider:

```
app.Use(async (context, next) =>
{
    Console.WriteLine("Before");

    await next(context);

    Console.WriteLine("After");
});
```

And:

```
app.Use(async (context, next) =>
{
    Console.WriteLine("Middleware 2 Before");

    await next(context);

    Console.WriteLine("Middleware 2 After");
});
```

Execution order is:

```
Middleware 1 Before
↓
Middleware 2 Before
↓
Controller / Endpoint
↓
Middleware 2 After
```





Middleware 1 After

This is important in interviews.

Middleware behaves somewhat like nested calls:

```
Middleware1
{
    Before

    Middleware2
    {
        Before

        Controller

        After
    }

    After
}
```



That is why middleware is useful for things like:

```
Performance measurement
Logging
Exception handling
Correlation IDs
Security headers
Request/response auditing
```



You can start a timer before `next()` and measure total execution after `next()` .

---

## Use , Run , and Map

Interviewers frequently ask this.

`Use()`

`Use()` normally continues to the next middleware.

```
app.Use(async (context, next) =>
{
    Console.WriteLine("Before request");
```

```
    await next();

    Console.WriteLine("After request");
});
```

Think:

Use = participate in pipeline + optionally continue



## Run()

Run() is terminal middleware.

```
app.Run(async context =>
{
    await context.Response.WriteAsync("Application terminated here.");
});
```

There is no next() .

Anything registered after this will normally not execute.

```
Use
↓
Use
↓
Run
X
Middleware below doesn't execute
```



## Map()

Map() creates a branch based on the request path.

```
app.Map("/admin", adminApp =>
{
    adminApp.Run(async context =>
    {
        await context.Response.WriteAsync("Admin Area");
    });
});
```

```
});  
});
```

For:

```
/admin  
/admin/users  
/admin/dashboard
```



the admin branch can execute.

Think:

```
Use → continue pipeline  
Run → terminate pipeline  
Map → create pipeline branch
```



---

## What is Custom Middleware?

Custom middleware is middleware created by us to implement application-specific cross-cutting functionality.

Typical production use cases include:

```
Global exception handling  
Request/response logging  
Correlation ID generation  
Performance monitoring  
Tenant resolution  
Custom authentication checks  
Security headers  
API auditing  
Culture/localization  
Request validation
```



A common custom middleware structure is:

```
public class RequestLoggingMiddleware  
{  
    private readonly RequestDelegate _next;  
    private readonly ILogger<RequestLoggingMiddleware> _logger;  
  
    public RequestLoggingMiddleware(  
        RequestDelegate next,
```

```

        ILogger<RequestLoggingMiddleware> logger)
    {
        _next = next;
        _logger = logger;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        _logger.LogInformation(
            "Request started: {Method} {Path}",
            context.Request.Method,
            context.Request.Path);

        await _next(context);

        _logger.LogInformation(
            "Request completed with status code {StatusCode}",
            context.Response.StatusCode);
    }
}

```

Register it:

```
</> C#
```



The critical line is:

```
</> C#
```



It transfers control to the next middleware.

Without it:

```

public async Task InvokeAsync(HttpContext context)
{
    await context.Response.WriteAsync("Stopped here");
}

```

the pipeline is short-circuited.

---

## Real Project Example: Global Exception Middleware

This is one of the best examples to give in a senior interview.

Instead of putting this everywhere:

```
try
{
    // logic
}
catch(Exception ex)
{
    // logging
}
```

we can centralize unexpected exception handling.

```
public class ExceptionMiddleware
{
    private readonly RequestDelegate _next;
    private readonly ILogger<ExceptionMiddleware> _logger;

    public ExceptionMiddleware(
        RequestDelegate next,
        ILogger<ExceptionMiddleware> logger)
    {
        _next = next;
        _logger = logger;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        try
        {
            await _next(context);
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, "Unhandled exception");

            context.Response.StatusCode =
                StatusCodes.Status500InternalServerError;

            context.Response.ContentType = "application/json";

            await context.Response.WriteAsJsonAsync(new
            {
                statusCode = 500,
                message = "An unexpected error occurred."
            });
        }
    }
}
```

```
    });  
  }  
}  
}
```

Register:

</> C#

Now:

```
Request  
↓  
Exception Middleware  
↓  
Controller  
↓  
Service  
↓  
Repository  
↓  
Database Exception  
↑  
Exception middleware catches it  
↓  
Logs exception  
↓  
Returns standard error response
```

The senior-level point is that you should **not expose stack traces or internal exception details to the client in production.**

## Real Project Example: Correlation ID Middleware

In distributed systems and microservices, a correlation ID is very useful.

Imagine:

```
React Application  
↓  
API Gateway  
↓  
Order API  
↓  
Payment API
```



↓  
Notification API

If something fails, logs from several services need to be connected.

Custom middleware can create or reuse a correlation ID:

```
public class CorrelationIdMiddleware
{
    private readonly RequestDelegate _next;

    public CorrelationIdMiddleware(RequestDelegate next)
    {
        _next = next;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        const string header = "X-Correlation-ID";

        var correlationId =
            context.Request.Headers[header].FirstOrDefault()
            ?? Guid.NewGuid().ToString();

        context.Response.Headers[header] = correlationId;

        await _next(context);
    }
}
```

Then logs could contain:

```
CorrelationId: 6ab43...
Order API started
Payment API called
Payment failed
Order API returned 500
```



This makes production troubleshooting much easier.

---

## Middleware Ordering

This is one of the most important experienced-level questions.

Middleware executes in the order in which it is registered.

For example:

```
app.UseExceptionHandler("/error");

app.UseHttpsRedirection();

app.UseRouting();

app.UseAuthentication();

app.UseAuthorization();

app.MapControllers();
```

Authentication must normally happen before authorization:

```
app.UseAuthentication();
app.UseAuthorization();
```

Because authentication answers:

Who are you?

Authorization answers:

Are you allowed to access this resource?

If authorization executes before authentication, `HttpContext.User` may not yet contain the authenticated identity.

---

## Why Exception Middleware Should Be Early

Suppose:

```
ExceptionMiddleware
  ↓
Authentication
  ↓
Authorization
  ↓
Custom Middleware
```

↓  
Controller

If anything downstream throws:

Controller exception  
Repository exception  
Authentication-related exception  
Custom middleware exception



the outer exception middleware can catch it.

This is why global exception handling is generally placed early in the pipeline.

---

## Short-Circuiting

An experienced candidate should know this term.

A middleware doesn't have to call:

</> C#



For example:

```
app.Use(async (context, next) =>
{
    if (!context.Request.Headers.ContainsKey("X-API-Key"))
    {
        context.Response.StatusCode =
            StatusCodes.Status401Unauthorized;

        await context.Response.WriteAsync("API key missing");

        return;
    }

    await next();
});
```

For a request without the API key:

Request  
↓



```
API-Key Middleware
↓
401 Unauthorized
X
Controller never executes
```

This is called **short-circuiting the middleware pipeline**.

## Middleware vs Filters

This is another common senior question.

Middleware works at the HTTP pipeline level:

```
Request
↓
Middleware
↓
Routing
↓
MVC
↓
Filters
↓
Controller
```



Filters operate primarily inside the MVC/action execution pipeline.

| Middleware                    | Filters                                 |
|-------------------------------|-----------------------------------------|
| Works on HTTP pipeline        | Works inside MVC/action pipeline        |
| Runs for many/all requests    | Associated with MVC/controllers/actions |
| Accesses HttpContext          | Has action/controller context           |
| Excellent for logging         | Excellent for action-specific concerns  |
| Exception handling            | Validation/action handling              |
| Authentication infrastructure | Authorization/action filters            |
| Correlation ID                | Result manipulation                     |

For example, I would use middleware for:

Correlation IDs  
Global exception handling  
Request logging  
Security headers



I might use filters for:

Action-specific validation  
Action execution logging  
Result manipulation  
Custom MVC authorization behavior



---

## Middleware Lifetime and Dependency Injection

Here's a more advanced point.

Conventional middleware is generally instantiated once, so be careful injecting scoped services directly into its constructor.

Instead of:

```
public MyMiddleware(  
    RequestDelegate next,  
    MyDbContext dbContext)
```

prefer resolving scoped dependencies through `InvokeAsync` :

```
public async Task InvokeAsync(  
    HttpContext context,  
    MyDbContext dbContext)  
{  
    // use dbContext  
  
    await _next(context);  
}
```

Alternatively, ASP.NET Core provides `IMiddleware` , which integrates middleware creation with DI lifetimes.

This is a useful senior-level detail because `DbContext` is normally scoped per request.

---

# Request Performance Middleware Example

Another good production example:

```
public async Task InvokeAsync(HttpContext context)
{
    var stopwatch = Stopwatch.StartNew();

    try
    {
        await _next(context);
    }
    finally
    {
        stopwatch.Stop();

        _logger.LogInformation(
            "{Method} {Path} executed in {ElapsedMilliseconds}ms",
            context.Request.Method,
            context.Request.Path,
            stopwatch.ElapsedMilliseconds);
    }
}
```

Now you might see:

```
GET /api/products → 35ms
GET /api/customers → 92ms
GET /api/reports → 4200ms
```



This immediately helps identify slow endpoints.

In a mature production environment, I'd generally integrate this with structured logging and observability rather than simply logging strings.

---

## UseMiddleware<T>() Extension Method

Instead of writing:

```
</> C#
```



directly everywhere, projects commonly create an extension:

```
public static class MiddlewareExtensions
{
    public static IApplicationBuilder
        UseCustomExceptionHandler(
            this IApplicationBuilder app)
    {
        return app.UseMiddleware<ExceptionHandler>();
    }
}
```

Then:

</> C#



This makes `Program.cs` cleaner and keeps configuration expressive.

---

## Interview Scenario

The interviewer may ask:

**“Your application has 50 controllers. You need to log every incoming request and execution time. Where would you implement it?”**

A strong answer:

I would implement this as middleware because request logging and execution timing are cross-cutting concerns. The middleware would capture request information and start a timer before calling the next delegate. After `await _next(context)` returns, I'd capture the status code and elapsed time. This avoids duplicating logging code across 50 controllers and gives us centralized observability.

Then explain:

```
var watch = Stopwatch.StartNew();

await _next(context);

watch.Stop();
```

## Another Interview Scenario

“How would you reject requests before they reach the controller?”

Answer:

Middleware can short-circuit the pipeline. I can inspect the request, and when validation fails, set the appropriate status code and return without invoking the next middleware.

Example:

```
if (!IsValidRequest(context))
{
    context.Response.StatusCode = 400;
    return;
}

await _next(context);
```

---

## Experienced-Level Interview Answer

For a 2-minute interview response, you can say:

**Middleware is a component in the ASP.NET Core HTTP request pipeline that can inspect, modify, process, or short-circuit requests and responses.**

The middleware pipeline is executed in the order configured in `Program.cs`. Each middleware receives an `HttpContext` and a `RequestDelegate` representing the next component.

Normally the middleware performs some logic, calls `await _next(context)`, and can perform additional work when the response travels back through the pipeline.

In production projects I've typically use middleware for cross-cutting concerns such as global exception handling, request/response logging, correlation IDs, performance monitoring, authentication infrastructure and security headers.

One important consideration is ordering. For example, authentication needs to run before authorization, while exception-handling middleware should generally be early enough to catch exceptions from downstream components.



Middleware can also short-circuit a request by not calling the next delegate—for example, rejecting an invalid API key before the request reaches the controller.

For controller/action-specific concerns, I would consider MVC filters instead of middleware.

## One line to remember

Middleware = Request/Response pipeline + cross-cutting concerns.  
Filter = MVC/Controller/Action pipeline + action-specific concerns.



And for the execution flow, remember:

```
Request
  ↓
Middleware 1 Before
  ↓
Middleware 2 Before
  ↓
Controller
  ↑
Middleware 2 After
  ↑
Middleware 1 After
  ↑
Response
```



That explanation is usually strong enough to handle both the basic question and the typical senior-level follow-ups about **ordering, short-circuiting, custom middleware, DI, exception handling, logging, and filters**.



Sources



Convert to PDF